AI in a Box: Portable Benchmarking and Mixed-Precision Compression for Edge LLM Deployment

Bhagya Patel
Student ID: 2565267

Abstract

Quantisation is the dominant practical mechanism for compressing large language models for local deployment: stored weights are reduced from higher-precision representations to lower-bit formats such as `Q4` or `Q8`, lowering model size and memory use at some cost to quality. Within the existing GGUF/`llama.cpp` ecosystem, this creates a deployment problem with no universally optimal choice: the best configuration depends on the device, the workload, and the deployment objective [2, 1, 3]. This paper studies that problem within the existing stack rather than through a new inference runtime. The first contribution is a portable benchmarking, orchestration, and recommendation framework that evaluates quantised model variants under explicit device and workload constraints, logs structured results, generates comparative plots, and returns concrete deployment recommendations. Developed on a MacBook Pro (M4 Pro, 24 GB), the same pipeline was then transferred unchanged at the runtime level to a Raspberry Pi 5, where it remained operational and produced objective-dependent recommendation outputs [10]. A further sanity test on `Llama-3.2-1B-Instruct-GGUF` required only configuration changes, supporting the claim that the framework is largely configuration-driven across GGUF-supported model families [7].

The second contribution is a tensor-level mixed-precision GGUF mixer that constructs runnable mixed models by combining a low-precision baseline with selected tensors from a higher-precision source. Motivated by activation-aware quantisation results showing that quantisation sensitivity is highly non-uniform [5], the framework improved mean perplexity from 9.069554 for vanilla `Q4_K_M` to 9.014643 by protecting the output layer and final two transformer blocks, and to 9.010936 with an activation-aware selector at matched budget. More importantly, a 500 MiB mixed model reached 8.903885 perplexity, outperforming uniform `Q6_K` at 8.911915 while remaining smaller on disk. Relative to the full `Q4`→`Q8` transition, that model recovered 83.2% of the perplexity gap while paying only 34.6% of the storage increase. Taken together, the results show that edge deployment should be treated as an empirical, device-specific systems problem, and that selective precision retention can recover much of the quality lost to aggressive quantisation without abandoning the existing local-inference stack.

1 Introduction

Running large language models on-device is attractive for privacy, offline availability, and reduced dependence on cloud infrastructure, but practical deployment is constrained by limited RAM, prompt latency, and strong hardware variability. In practice, the dominant compression lever is quantisation: higher-precision model weights are stored in lower-bit formats to reduce disk footprint and memory pressure, but this typically degrades quality to some degree. Within the GGUF/`llama.cpp` ecosystem studied here, deployment is therefore not simply a question of whether a model fits in memory. A model that technically fits on a device may still be unsuitable if prompt ingestion is too slow, time-to-first-token is too high for interactive use, or the chosen quantisation sacrifices more quality than the use case can tolerate. Edge deployment is therefore a systems problem rather than only a modelling problem.

This project addresses that problem in two complementary stages. Part 1 develops a portable benchmarking and recommendation framework for GGUF-based local inference with `llama.cpp` [2, 1]. Rather than relying on static rules such as "always choose the smallest quant that fits", the framework measures candidate configurations on the target device and recommends the most suitable configuration for the stated deployment objective under explicit constraints such as memory ceiling, throughput requirement, latency target, and context length. Given a target device and deployment objective, the framework turns local LLM selection into a data-driven procedure rather than trial-and-error. It also supports different operational priorities, including latency-first, quality-first, and maximum-context scenarios; the last of these is particularly relevant to retrieval-augmented (RAG) settings, where maximising the feasible prompt budget for larger retrieved documents may matter more than maximising raw decode speed.

Part 2 addresses the quality side of the same trade-off by constructing runnable mixed-precision GGUF models

that selectively preserve higher precision for chosen tensors while leaving the remainder of the model in a lower-bit baseline. The motivation came in part from activation-aware quantisation work such as AWQ, which argues that quantisation sensitivity is not uniformly distributed across an LLM and that protecting a small salient subset can substantially reduce quantisation error [5]. That observation motivated the activation-aware mixed-precision strategy explored here: rather than treating all tensors as equally deserving of higher precision, the framework asks whether selective tensor promotion can recover disproportionately more quality for a bounded storage budget.

The work is deliberately practical. It does not propose a new inference runtime or a new quantisation algorithm. Instead, it asks what can be achieved within a realistic local-inference stack already used in practice: GGUF for model interchange and `llama.cpp` for execution and benchmarking [2, 1, 3]. This keeps the contributions deployable and reproducible.

The main contributions are:

- a portable benchmarking, orchestration, and recommendation pipeline for GGUF/`llama.cpp` deployments under explicit edge constraints;
- an empirical portability validation of that pipeline across accelerated and CPU-only runtime profiles, a physical Raspberry Pi 5 target, and an additional GGUF model family using configuration changes only;
- a tensor-level mixed-precision GGUF mixer, extended with activation-aware budgeted selection, that improves the quality–storage trade-off while remaining compatible with the same runtime ecosystem.

The two parts were designed with different priorities. Part 1 prioritised practical portability and deployment utility, whereas Part 2 prioritised a focused research study of selective precision allocation within a fixed runtime ecosystem. This distinction is important when interpreting the scope of the portability claims made later in the paper.

2 Method

The project is implemented as two connected but methodologically distinct pipelines. Part 1 is a portable benchmarking, orchestration, and recommendation framework for local GGUF deployment. Part 2 is a tensor-level mixed-precision GGUF construction and evaluation framework built on the same runtime ecosystem, but addressing a different question: once a low-bit model has been selected for deployment efficiency, how much quality can be recovered by selectively promoting only a subset of tensors to higher precision? The two parts are linked by deployment logic but differ in emphasis: Part 1 prioritises measurement and recommendation under hardware constraints, whereas Part 2 prioritises selective precision allocation and controlled quality–storage analysis.

2.1 Part 1: Benchmarking and Recommendation

Part 1 addresses the following practical deployment question: given a target device, a candidate GGUF model family, and user-defined constraints such as memory budget, latency target, and required throughput, which configuration should actually be deployed? In this project, a *configuration* is defined by a model variant (for example `Q3_K_M`, `Q4_K_M`, `Q6_K`, or `Q8_0`), a runtime profile (for example CPU-only or accelerated execution), and a workload description in terms of prompt and generation budgets. The Part 1 framework was designed to answer that question empirically rather than heuristically: instead of assuming that the smallest model that fits is automatically the best choice, it measures candidate configurations on the target device and then selects among the feasible ones under an explicit recommendation policy.

The implementation is configuration-driven. A run configuration specifies the model repository, the GGUF filenames to evaluate, the device/runtime profiles, the thread count, generation length, and the prompt budgets to sweep. A separate constraints configuration specifies the deployment scenario to optimise for, including the target device, maximum RSS, desired headroom, maximum TTFT proxy, minimum generation throughput, and objective function. Example YAML files for these inputs are provided in the project's `configs/` directory. This separation was intentional. It distinguishes the *experiment grid* from the *deployment decision*, so that a broad benchmark sweep can be reused across multiple recommendation scenarios without rerunning the full experiment.

At the code level, orchestration is centred on `bench/pipeline.py`, which coordinates model download, benchmark execution, ingestion, plotting, and recommendation. Missing GGUF files are resolved by `bench/download_models.py`, while `bench/run_grid.py` expands candidate quantisations, device profiles, and prompt budgets into concrete run specifications with deterministic output paths. This stage also supports rerun robustness by skipping runs whose expected `.jsonl` and `.time` artefacts already exist and are non-empty.

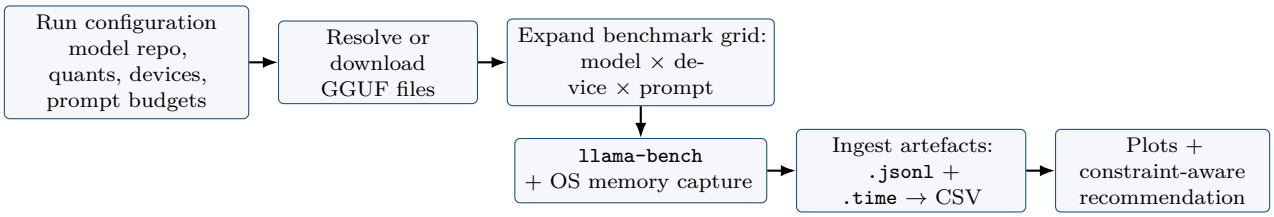


Figure 1: Part 1 pipeline. The workflow separates run-time experiment specification from deployment constraints: GGUF candidates are resolved, the benchmark grid is expanded, `llama-bench` is executed with memory capture, per-run artefacts are ingested into a unified CSV, and that table is then reused for plotting and constraint-aware recommendation.

Each configuration is executed through `llama-bench` with JSONL output enabled. The framework does not modify `llama.cpp`; it treats it as a fixed execution engine and layers the experimental logic above it [2, 1, 3]. For every run, structured benchmark output is paired with operating-system memory capture, producing a `.jsonl` artefact from `llama-bench` and a matching `.time` file containing peak RSS. On macOS, RSS was collected with `/usr/bin/time -l`; on Linux-based systems such as the Raspberry Pi, the same interface was preserved using `/usr/bin/time -v`. The artefacts are then merged by `bench/ingest_jsonl_time.py` into `results/runs.csv`, which records model identity, quantisation, device/backend, thread count, GPU-offload setting, prompt and generation budgets, `pp_tok_s`, `tg_tok_s`, peak RSS, and build metadata. It also computes a time-to-first-token proxy from the measured throughput terms:

$$\mathrm{TTFT}_{\mathrm{proxy}} = \frac{n_{\mathrm{prompt}}}{\mathrm{pp_tok_s}} + \frac{1}{\mathrm{tg_tok_s}}, \tag{1}$$

which approximates the time required to ingest the prompt and emit the first generated token. The unified CSV then serves as the central intermediate representation for both empirical analysis and automated selection.

Recommendation is implemented in `bench/recommend.py` and invoked after ingestion. The procedure is deliberately simple and explicit. First, the CSV is filtered to the relevant deployment scenario: device, generation budget, and, for most objectives, a target prompt budget. Second, hard constraints remove infeasible candidates, including maximum RSS, optional memory headroom, maximum TTFT proxy, minimum generation throughput, and, where relevant, minimum context length. Only then are the remaining configurations ranked. This two-stage structure reflects the actual deployment problem: a configuration that is fast but violates the memory budget, or that fits in memory but is too slow for interactive use, should not remain under consideration.

Four ranking objectives are supported: `speed_first`, `quality_first`, `balanced`, and `max_context`. The first prioritises generation speed, the second prefers higher-ranked quantisation among feasible options, the third trades off speed, latency, and memory, and the fourth prefers the largest feasible prompt budget before considering secondary criteria. For the balanced objective, the score used in this study is

$$\mathrm{score} = 1.0 \cdot \mathrm{tg_tok_s} - 0.3 \cdot \mathrm{TTFT}_{\mathrm{proxy}} - 1.0 \cdot \mathrm{RSS}_{\mathrm{GB}}. \tag{2}$$

The importance of this score is not that it is the unique correct objective, but that it makes the deployment policy explicit and inspectable.

A further refinement is pruning. To avoid wasting time on clearly dominated long-context runs, the pipeline groups configurations by model and device, evaluates prompt sizes in ascending order, and stops exploring larger prompts when memory already exceeds the effective budget, the next-prompt TTFT is predicted to violate the configured threshold, or a run times out. Skipped configurations are written to `skipped_runs.csv` rather than being treated as valid measurements. The overall result is a portable, measurement-driven deployment framework implemented above an unchanged `llama.cpp` runtime: not a new inference engine, but a reusable evaluation and decision layer that turns low-level benchmark traces into explicit deployment recommendations under edge-oriented constraints.

2.2 Part 2: Mixed-Precision GGUF Mixing Framework

Part 2 addresses a different but closely related question to Part 1. Once a low-bit model has been selected because it satisfies an edge deployment envelope, some quality has typically already been sacrificed relative to a higher-precision variant. Uniformly moving the whole model to a larger quant or floating-point representation is the simplest way to recover quality, but it also incurs the full storage and runtime cost of that higher-precision model. The aim here is to test whether some of that lost quality can instead be recovered *selectively*, by preserving higher precision only for a subset of tensors while leaving the remainder of the model at the lower-precision baseline.

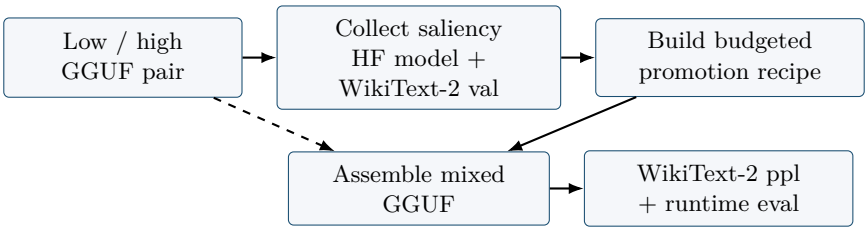


Figure 2: Part 2 pipeline. Compatible low/high GGUF sources are analysed through activation-aware calibration on WikiText-2 raw validation text, converted into a budgeted promotion recipe, assembled into a runnable mixed GGUF, and then evaluated by perplexity and runtime within the same ecosystem as Part 1.

The framework therefore operates on *pairs* of compatible GGUF files for the same base model: a low-precision baseline and a higher-precision source. In the experiments reported here, this pair is `Qwen2.5-3B-Instruct` in `Q4_K_M` and `Q8_0` form, but the core assembly mechanism is not specific to Qwen. At a high level, the method has six stages: (i) choose the low/high GGUF pair, (ii) collect saliency statistics from a calibration pass, (iii) filter valid candidate tensors, (iv) construct a budgeted promotion recipe, (v) assemble a runnable mixed GGUF, and (vi) evaluate the result using the same `llama.cpp`-based harness as Part 1. The main scripts implementing these stages are `collect_saliency.py`, `build_budgeted_recipe.py`, and `mix_gguf.py`.

The design choice is deliberately **tensor-level** rather than within-tensor mixed precision. Each tensor is taken in its entirety either from the low-precision GGUF or from the higher-precision GGUF; the framework does not attempt channel-level, row-level, or element-level mixing. This restriction is important. It keeps the method compatible with an unmodified GGUF/`llama.cpp` runtime and makes the output directly executable, but it also means that selection granularity is coarser than in methods that alter quantisation kernels or introduce custom inference logic. The contribution is therefore not a new quantiser or runtime, but a practical assembly and evaluation framework for selective precision allocation inside an existing local-inference stack.

The central engineering component is the mixer itself, implemented in `mix_gguf.py`. Given a configuration specifying the low-precision GGUF, the higher-precision GGUF, an output path, and a tensor-selection policy, the mixer first validates that the two sources are structurally compatible. This includes checking model architecture metadata, tensor-name agreement, and tensor-shape agreement. Only once these checks pass does assembly proceed. The mixer then iterates through the tensor list and, for each tensor name, decides whether it should be copied from the low source or the high source. In conceptual terms, the mixer implements the mapping

$$\texttt{tensor_name} \mapsto \{\texttt{low}, \texttt{high}\}, \tag{3}$$

where the choice is supplied by either a heuristic policy or an explicit recipe.

A crucial implementation detail is that GGUF tensors cannot always be reconstructed safely from their logical matrix shape alone. For quantised tensors in particular, the file stores a packed representation whose raw layout differs from the logical tensor dimensions exposed at model level. Early versions of the mixer failed because they attempted to write quantised tensors using logical shape information rather than the raw packed tensor layout expected by the GGUF writer. The final implementation instead copies the raw tensor payload directly from the chosen source GGUF and writes it back using the tensor's stored packed shape and tensor type. This proved essential: without preserving the packed representation, the output file may be syntactically writable but will not be a valid runnable GGUF for `llama.cpp`. In other words, correctness of the mixed model depends not only on choosing the right source tensor, but on preserving the exact on-disk representation expected by the GGUF format [1, 2].

The first tensor-selection strategy studied was deliberately structural and heuristic. The motivating assumption was that quantisation sensitivity is unlikely to be uniformly distributed across the network, and that tensors closer to the final token-distribution computation may matter disproportionately. Three simple protected sets were therefore evaluated: the final transformer block, the final two transformer blocks, and the output projection plus the final two blocks. These policies required no calibration data and provided an interpretable starting point for the study. They were implemented directly inside the mixer as named policies over tensor names, which made it possible to build mixed models immediately once a compatible low/high GGUF pair was available.

The second strategy retained the same mixing and assembly path but replaced the tensor-selection stage with an activation-aware, budgeted recipe construction process. This stage is split across two scripts. The first, `collect_saliency.py`, performs a calibration pass using the Hugging Face model rather than the GGUF runtime. It loads the original `Qwen/Qwen2.5-3B-Instruct` model, registers forward hooks on a set of modules of interest, and records activation statistics over held-out calibration text drawn from the WikiText-2 raw

validation split [6, 4, 9]. In the current implementation, this hook mapping is **Qwen-specific**: Hugging Face module names such as self-attention projections and feed-forward projections are mapped manually to the corresponding GGUF tensor names used later by the mixer. This part of the pipeline is therefore model-family specific, even though the downstream mixer is generic. The calibration script records statistics such as mean absolute activation, activation RMS, count, and weight RMS, and writes them to a saliency JSON artefact. Figure 3 summarises the resulting per-block, per-family saliency field using the same score later used in budgeted recipe construction. [5, 8]

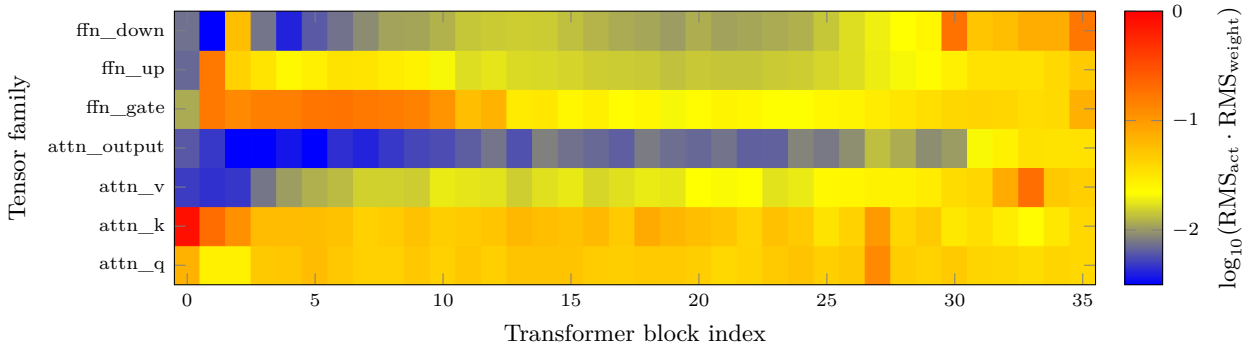


Figure 3: Activation-aware saliency heatmap for the Qwen2.5-3B-Instruct calibration pass. Each cell shows the log-scaled tensor score used by the budgeted selector, $\mathrm{RMS}_{\mathrm{activation}} \times \mathrm{RMS}_{\mathrm{weight}}$, aggregated at the level of GGUF tensor families within each transformer block. Warmer colours indicate higher saliency scores and therefore larger promotion value under the activation-aware recipe construction stage.

Figure 3 visualises the saliency signal later used for activation-aware tensor promotion. The quantity shown is not raw activation RMS alone, but the log-scaled product $\mathrm{RMS}_{\mathrm{activation}} \times \mathrm{RMS}_{\mathrm{weight}}$, matching the score used by the budgeted selector. Higher values in this score appear as warmer colours (yellow to red), while lower values appear as cooler colours (blue). The resulting pattern is clearly non-uniform across both depth and tensor family. In particular, the earliest layers contain unusually large saliency for `attn_k`, while several early feed-forward families are also prominent; later blocks show relatively stronger `attn_v` and especially `ffn_down` responses. This supports the view that quantisation sensitivity is distributed unevenly across the model rather than concentrated in a single contiguous suffix. At the same time, the concentration of many stronger cells toward later blocks helps explain why suffix-based structural protection already formed a strong heuristic baseline before the activation-aware selector was introduced.

The second activation-aware stage, implemented in `build_budgeted_recipe.py`, turns those saliency measurements into a concrete promotion recipe. The script reads the low-precision GGUF, the higher-precision GGUF, and the saliency JSON, then constructs a set of candidate tensors subject to several filters. A tensor is considered eligible only if it exists in both source files, has matching shape, is at least two-dimensional, has valid saliency statistics, and yields a positive storage delta when promoted from the low source to the high source. These filters separate the generic idea of "importance" from the concrete engineering question of whether promotion is both meaningful and representable in a valid mixed GGUF.

For each eligible tensor t, a saliency score is then computed as

$$\mathrm{score}(t) = \mathrm{RMS}_{\mathrm{activation}}(t) \times \mathrm{RMS}_{\mathrm{weight}}(t). \tag{4}$$

This is not claimed to be the uniquely correct notion of tensor importance; rather, it is a simple activation-aware proxy that combines observed response magnitude with a property of the stored weight tensor itself. The selector used in the final experiments also forced inclusion of `output.weight`, because the earlier heuristic experiments had already shown that protecting the output projection was consistently beneficial. Given a target additional storage budget over the Q4 baseline, recipe construction is posed as a budgeted selection problem in which each tensor has value $\mathrm{score}(t)$ and cost equal to its promotion byte delta. The implementation discretises storage into 0.5 MiB bins, applies a dynamic-programming 0–1 knapsack over the remaining budget, and then greedily fills any residual capacity by score-per-byte. Algorithm 1 summarises this procedure.

The resulting recipe is then passed back to `mix_gguf.py`, which supports an `explicit_list` policy in addition to the heuristic structural policies. This separation of concerns is deliberate: saliency collection and budgeted selection decide *which* tensors should be protected, while the mixer decides *how* to assemble a correct GGUF from those decisions. The assembly core therefore remains reusable even if the tensor-selection policy changes.

The final stage is evaluation. Once assembled, each mixed model is treated exactly like any other GGUF deployment candidate. Perplexity is measured with `llama-perplexity` on a fixed held-out evaluation corpus

Algorithm 1 Budgeted tensor promotion by forced inclusion, knapsack selection, and greedy fill

Input: low/high GGUF tensor maps, saliency JSON, target extra budget B, forced set F
Output: explicit promotion recipe

Build candidate set C of tensors present in both sources with matching shape, dimension ≥ 2, valid saliency statistics, and positive byte delta
for all $t \in C$ **do**
    $\text{score}(t) \leftarrow \text{RMS}_{\text{activation}}(t) \cdot \text{RMS}_{\text{weight}}(t)$
    $\text{cost}(t) \leftarrow \lceil \Delta\text{bytes}(t)/(0.5\,\text{MiB}) \rceil$
$C_F \leftarrow \{t \in C : t.\text{name} \in F\}$, $C_R \leftarrow C \setminus C_F$
Subtract the forced byte cost from B; abort if the remainder is negative
Run a 0–1 knapsack over C_R with value $\text{score}(t)$ and bin cost $\text{cost}(t)$
Recover the selected set S from the dynamic-programming table
for all $t \in C_R \setminus S$ in descending order of $\text{score}(t)/\Delta\text{bytes}(t)$ **do**
    if t fits in the remaining byte budget **then**
        add t to S
Output the recipe for $C_F \cup S$ as an explicit tensor-name list

derived from the WikiText-2 raw test split, and selected models are benchmarked with `llama-bench` using the same runtime settings and metric definitions introduced in Part 1 [2, 6, 4]. This reuse of the Part 1 harness is methodologically important. It means that the mixed models are not evaluated in a separate bespoke environment, but in the same stack used for deployment benchmarking, making the quality–efficiency trade-off directly comparable.

The generic and model-specific parts of Part 2 can therefore be separated clearly. The *generic* component is the GGUF assembly core: if two source GGUF files are compatible in architecture, tensor naming, and tensor shapes, the mixer can in principle construct a runnable mixed model from any policy that specifies which tensors should be drawn from which source. The *model-specific* component is the saliency-collection layer used in the activation-aware experiments, because it depends on a manual mapping from Hugging Face module names to GGUF tensor names for the chosen architecture. In this report, Part 2 should therefore be interpreted as a research-oriented mixed-precision framework with a generic assembly mechanism and a Qwen-specific activation-aware selector developed to study the quality–storage trade-off in a concrete setting.

The result is a pipeline that starts from a deployment-efficient low-bit baseline, constructs budgeted mixed-precision upgrades within the same runtime ecosystem, and evaluates whether selective precision allocation yields a better quality–efficiency operating point than uniform quantisation alone.

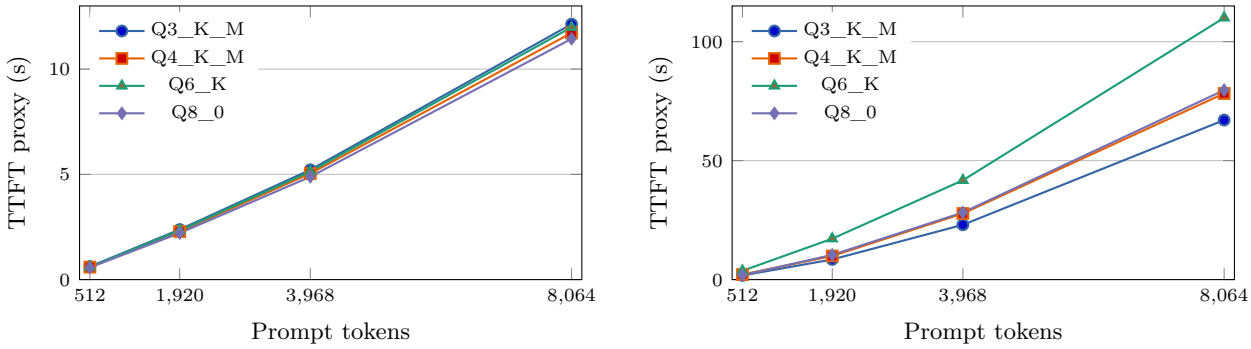
3 Experimental Setup

Part 1 used `Qwen2.5-3B-Instruct` in GGUF form with quantised variants `Q3_K_M`, `Q4_K_M`, `Q6_K`, and `Q8_0` [9, 11, 3]. The Mac corpus used 8 threads, 1 repetition, prompt lengths {512, 1920, 3968, 8064}, generation length 128, and two runtime profiles: accelerated `MTL0` with `-ngl 80` and CPU-only `BLAS` with `-ngl 0`. The Raspberry Pi 5 validation used CPU-only `BLAS`, 4 threads, 1 repetition, prompt lengths {128, 256, 512, 1024, 1536}, generation length 128, and the candidate variants `Q3_K_M`, `Q4_K_M`, and `Q6_K` [10].

Part 2 used `Q4_K_M` as the low-precision baseline and `Q8_0` as the higher-precision source. Perplexity was measured with `llama-perplexity` on a held-out file derived from the WikiText-2 raw test split, while activation-aware saliency collection used text drawn from the WikiText-2 raw validation split, capped at 64 chunks of 256 tokens [6, 4]. Runtime evaluation for the mixed models used the fixed `MTL0` setting with 8 threads, `-ngl 80`, prompt length 1920, generation length 128, and 1 repetition.

Table 1: Compact summary of the main experimental settings.

Component	Setting
Part 1 Mac corpus	`Q3_K_M`, `Q4_K_M`, `Q6_K`, `Q8_0`; `MTL0` and `BLAS`; 8 threads
Mac prompt sweep	{512, 1920, 3968, 8064} with generation length 128
Raspberry Pi validation	`Q3_K_M`, `Q4_K_M`, `Q6_K`; `BLAS`; 4 threads
Pi prompt sweep	{128, 256, 512, 1024, 1536} with generation length 128
Part 1 metrics	`pp_tok_s`, `tg_tok_s`, TTFT proxy, peak RSS
Part 2 source pair	`Q4_K_M` baseline and `Q8_0` source
Part 2 quality metric	Mean perplexity on WikiText-2 raw test split
Part 2 calibration	WikiText-2 raw validation split, max 64 chunks of 256 tokens
Part 2 runtime setting	`MTL0`, 8 threads, `-ngl 80`, `1920/128`



(a) Mac `MTL0` full sweep.

(b) Mac `BLAS` full sweep.

Figure 4: Full Mac benchmark corpus. Acceleration compresses prompt latency across the full context sweep. Under `MTL0` the TTFT curves remain tightly clustered, whereas under CPU-only `BLAS` they separate sharply and expose objective-dependent, non-monotonic behaviour across quantisations.

Table 2: Representative Mac operating point at `1920/128`. Even on the same machine, the accelerated and CPU-only profiles induce different trade-offs across prompt speed, generation speed, and memory.

Device	Quant	PP tok/s	TG tok/s	TTFT proxy (s)	RSS (MB)
MTL0	Q3_K_M	812.07	67.26	2.379	2064.8
MTL0	Q4_K_M	844.70	83.94	2.285	2425.5
MTL0	Q6_K	827.97	68.47	2.334	3082.2
MTL0	Q8_0	875.62	62.00	2.209	3938.6
BLAS	Q3_K_M	227.45	60.00	8.458	2293.7
BLAS	Q4_K_M	194.07	75.40	9.907	2534.7
BLAS	Q6_K	111.73	51.11	17.204	3191.3
BLAS	Q8_0	185.19	55.93	10.385	4048.7

4 Results and Discussion

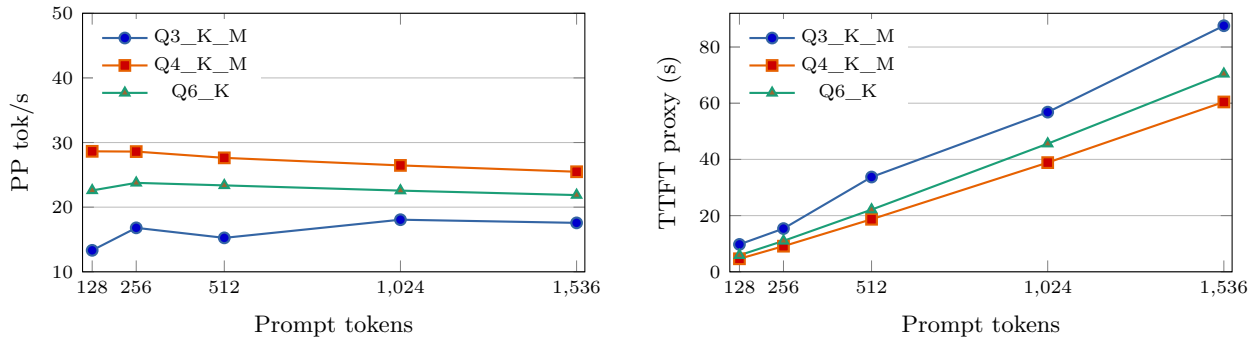
4.1 Cross-Device Benchmarking and Raspberry Pi Validation

The Mac sweeps first establish that runtime profile is a first-order deployment variable rather than a minor implementation detail. Under accelerated `MTL0`, all four quantisations remained usable across the studied prompt range, with TTFT proxy staying below 12 s even at 8064 tokens. Under CPU-only `BLAS`, the same model family behaved very differently: TTFT rose to 67.046 s for `Q3_K_M`, 78.226 s for `Q4_K_M`, 79.614 s for `Q8_0`, and 110.027 s for `Q6_K` at 8064 tokens. This is already enough to show that deployment choice cannot be reduced to quantisation alone; it depends jointly on hardware, prompt length, and objective.

A more revealing view is the representative `1920/128` operating point in Table 2. On `MTL0`, the latency spread is small: TTFT ranges only from 2.209 s to 2.379 s, even though RSS rises from 2064.8 MB for `Q3_K_M` to 3938.6 MB for `Q8_0`. At that operating point, `Q8_0` gives the lowest TTFT proxy through faster prompt ingestion, whereas `Q4_K_M` gives the highest generation throughput at 83.94 tok/s. In other words, even on the same host, the preferred quant depends on whether prompt latency, decode speed, memory headroom, or quality is being prioritised. Under `BLAS`, that tension is stronger: `Q3_K_M` gives the lowest TTFT proxy at 8.458 s, but `Q4_K_M` produces the fastest generation at 75.40 tok/s. The objective-dependent ordering is therefore visible before moving to a second device class.

Memory behaviour also differs materially across runtime profiles. For example, `Q8_0` under `MTL0` rises from 3880.5 MB at 512 tokens to 4178.7 MB at 8064 tokens, whereas under `BLAS` it rises from 3944.9 MB to 5231.1 MB across the same sweep. The accelerated profile therefore improves more than latency alone: it also makes long-context operation substantially less memory-expensive. The Mac corpus is thus useful not only as a benchmark baseline, but as evidence that the same model family can occupy quite different operating regimes depending on runtime profile.

The Raspberry Pi results are the core portability result. After initial development on the MacBook, the same Python-side benchmark–recommendation framework was transferred to a Raspberry Pi 5 and remained operational without modifying `llama.cpp` [10, 2]. The system executed a structured benchmark sweep, produced the same `.jsonl+.time` artefact pattern, ingested those logs into the unified CSV, generated plots, and returned recommendation outputs. This is stronger than simply showing that one model can “run” on a Pi: it demonstrates that the full benchmark–recommendation workflow transfers across hardware classes.



(a) Prompt-processing throughput.

(b) Latency scaling on Raspberry Pi 5.

Figure 5: Raspberry Pi 5 validation sweep. `Q4_K_M` consistently dominates prompt ingestion on the Pi, while `Q3_K_M` retains the highest generation speed. The balanced recommendation therefore shifts once prompt latency becomes important.

Table 3: Scenario-specific Raspberry Pi 5 recommendations derived from the same benchmark corpus.

Objective	Workload	Recommended quant	TG tok/s	TTFT proxy (s)	RSS (MB)
Speed-first	256 / 128	Q3_K_M	7.29	15.380	2477.5
Balanced	512 / 128	Q4_K_M	6.03	18.698	3899.5
Quality-first	512 / 128	Q6_K	4.62	22.120	5136.0
Max-context	1536 / 128	Q6_K	4.69	70.444	5136.0

The Pi sweep also makes the mechanism behind the recommendations clearer than the Mac curves alone. Across the full {128, 256, 512, 1024, 1536} prompt sweep, `Q4_K_M` consistently achieved the fastest prompt-processing throughput, while `Q3_K_M` consistently gave the highest generation throughput. At 512 prompt tokens, for example, `Q4_K_M` processed prompts at 27.63 tok/s versus 15.25 tok/s for `Q3_K_M`, a substantial advantage in the prefill-dominated part of TTFT, while generation throughput differed much less (6.03 versus 6.29 tok/s). The result was a TTFT proxy of 18.698 s for `Q4_K_M` against 33.733 s for `Q3_K_M`. That gap widens with context: by 1536 tokens, `Q4_K_M` remains at 60.444 s while `Q3_K_M` reaches 87.576 s. The practical lesson is that the smallest model is not automatically the best latency choice once prompt ingestion, rather than only decode speed, becomes important.

Memory on the Pi is also informative. Within the studied sweep, RSS was comparatively flat for a given quantisation; for example, `Q4_K_M` remained at 3899.5 MB from 128 to 1536 prompt tokens. This means that, in the examined range, the balanced recommendation is driven primarily by throughput and latency rather than by rapidly increasing context memory. `Q6_K` is then chosen not because it is more efficient in systems terms—it is slower and substantially heavier than `Q4_K_M`—but because a quality-biased policy can still rationally prefer the higher-precision option once the deployment envelope permits it.

Table 3 is central to the practical value of the framework. The same Pi benchmark corpus produced different recommendations when the objective changed. For a short-context latency-first use case, the recommender selected `Q3_K_M`, reflecting its faster decode and much lower memory use. For a balanced 512-token workload, it selected `Q4_K_M`, because prompt-processing throughput had become the dominant factor in perceived latency. For quality-first and maximum-context policies, it selected `Q6_K`, accepting higher memory and weaker systems performance in order to prefer the higher-precision operating point. The recommendations are therefore not arbitrary outputs of a fixed rule: they are materially different decisions induced by the same measured corpus under different explicit objectives.

The repeatability check in Table 4 adds a useful robustness result. For `Q3_K_M` and especially `Q4_K_M`, variance was very low across all reported metrics, supporting the use of single-run sweeps for broad comparative benchmarking under constrained runtime budgets. `Q6_K` was less stable, with noticeably higher variability in both speed and observed RSS. In particular, the large RSS standard deviation was driven by a spread from 4309.0 MB to 5136.0 MB across the three runs. This does not invalidate the broader trend, but it does suggest

Table 4: Raspberry Pi 5 repeatability study at `512/128` over three runs (mean ± standard deviation).

Quant	PP tok/s	TG tok/s	TTFT proxy (s)	RSS (MB)
Q3_K_M	18.69 ± 0.28	7.28 ± 0.03	27.53 ± 0.41	2477.3 ± 0.35
Q4_K_M	27.66 ± 0.05	6.06 ± 0.02	18.68 ± 0.04	3899.3 ± 0.29
Q6_K	22.87 ± 0.89	4.58 ± 0.18	22.63 ± 0.90	4860.1 ± 477.2

Table 5: Configuration-driven portability sanity test on `Llama-3.2-1B-Instruct-GGUF`. Results are taken from the framework's balanced recommendation query for an `MTL0 1920/128` workload.

Quant	PP tok/s	TG tok/s	TTFT proxy (s)	RSS (MB)
Q4_K_M	2305.00	197.61	0.838	1360.6
Q6_K	2267.91	166.66	0.854	1565.0
Q8_0	2390.10	150.88	0.807	1971.5

Table 6: Selected mixed-precision and uniform-quant results under the fixed `MTL0` benchmark configuration (`1920/128`, 8 threads, `-ngl 80`).

Model	Mean PPL	TG tok/s	TTFT proxy (s)	RSS (MB)
Vanilla Q4_K_M	9.069554	83.71	2.288	2425.6
Output + Last 2 Blocks Q8 (≈132 MiB)	9.014643	81.36	2.286	2631.7
Activation-aware, matched budget (≈132 MiB)	9.010936	81.26	2.286	2630.1
Activation-aware, 500 MiB budget	8.903885	73.99	2.261	2995.8
Vanilla Q6_K	8.911915	68.47	2.334	3082.2
Vanilla Q8_0	8.870479	61.98	2.210	3938.7

that heavier configurations should be repeated when deployment margins are tight.

4.2 Configuration-Driven Portability Across Model Families

To test whether Part 1 was tied too closely to a single model family, the pipeline was re-run on `Llama-3.2-1B Instruct GGUF` using only configuration changes [7]. No code changes were required: the same framework successfully performed model download, benchmark orchestration, log ingestion, plot generation, and recommendation. This supports the claim that Part 1 is largely configuration-driven across GGUF-supported model families, rather than being hard-coded to `Qwen2.5-3B-Instruct` [9, 7].

Under a balanced `1920/128` workload on `MTL0`, the recommender selected `Q4_K_M`. The top feasible candidates returned by the framework are shown in Table 5. The selection is again informative rather than trivial: `Q8_0` gives the lowest TTFT proxy at 0.807 s, but `Q4_K_M` offers substantially faster generation throughput at 197.61 tok/s versus 150.88 tok/s, while also using less memory (1360.6 MB versus 1971.5 MB). The balanced recommendation therefore reflects the same principle seen earlier: the framework does not simply choose the highest quant or the lowest TTFT row, but applies an explicit policy over measured trade-offs. Although this was only a sanity test rather than a second full benchmark campaign, it strengthens the portability claim in a practically meaningful way: the same end-to-end workflow transferred across device profiles, operating environments, and model families.

4.3 Mixed-Precision Compression Results

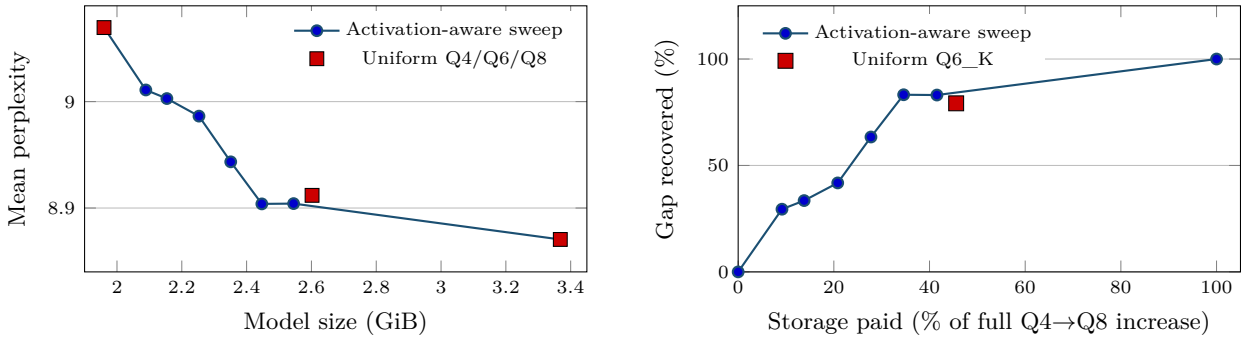
The mixed-precision experiments first established feasibility: all heuristic recipes produced runnable GGUF files that could be loaded directly by `llama.cpp` [2, 1]. Protecting only the final transformer block improved mean perplexity over the `Q4_K_M` baseline, protecting the final two blocks improved it further, and protecting the output projection plus the final two blocks gave the strongest structural result at 9.014643. This is a useful result in its own right: it shows that later layers and the output projection are disproportionately valuable places to preserve precision, even before any calibration-driven selection is introduced.

Table 6 makes two further points clear. First, the structural heuristic is already strong. Moving from vanilla `Q4_K_M` to the output-plus-last-two-blocks recipe improves mean perplexity by 0.054911 while leaving TTFT effectively unchanged (2.288 s versus 2.286 s) and reducing generation throughput only modestly (83.71 to 81.36 tok/s). The matched-budget activation-aware selector then improves slightly further to 9.010936. The gain over the best heuristic is small rather than dramatic, but that is precisely why it is meaningful: the structural baseline was already competitive, so the activation-aware method is refining a strong starting point rather than rescuing a weak one.

Second, the larger-budget mixed model is the main operating-point result. At a 500 MiB promotion budget, mean perplexity falls to 8.903885. Relative to uniform `Q6_K`, this model is better across all four reported metrics: lower perplexity (8.903885 versus 8.911915), higher generation throughput (73.99 versus 68.47 tok/s), lower TTFT proxy (2.261 versus 2.334 s), and lower RSS (2995.8 versus 3082.2 MB). It also remains smaller on disk (2.447 GiB versus 2.602 GiB). This is a stronger claim than simply saying that the mixed model "works": at the studied operating point, selective promotion is competitive not only in quality but in systems terms as well.

Table 7: Activation-aware budget sweep. Storage is reported in GiB from exact file byte counts. "Storage paid" denotes the fraction of the full `Q4_K_M`→`Q8_0` storage increase incurred by each model, while "gap recovered" denotes the fraction of the full `Q4_K_M`→`Q8_0` perplexity gap recovered.

Model	Size (GiB)	Mean PPL	Storage paid (%)	Gap recovered (%)
Vanilla Q4_K_M	1.960	9.069554	0.00	0.00
Act.-aware (≈132 MiB)	2.089	9.010936	9.16	29.45
Act.-aware (200 MiB)	2.154	9.002860	13.79	33.50
Act.-aware (300 MiB)	2.253	8.986345	20.81	41.80
Act.-aware (400 MiB)	2.351	8.943408	27.76	63.37
Act.-aware (500 MiB)	2.447	8.903885	34.59	83.22
Act.-aware (600 MiB)	2.545	8.904166	41.55	83.08
Vanilla Q8_0	3.368	8.870479	100.00	100.00



(a) Quality as a function of size, with `Q6_K` as a uniform-quant anchor.

(b) Gap recovered versus storage paid, showing the non-linear frontier.

Figure 6: Mixed-precision quality–storage trade-offs. The frontier improves rapidly at modest budgets, the 500 MiB mixed model slightly outperforms uniform `Q6_K` while using less storage, and gains flatten beyond that point.

To characterise the quality–storage frontier more systematically, activation-aware mixed models were then constructed under a sequence of additional storage budgets over the `Q4_K_M` baseline. Table 7 reports the exact values underlying this sweep, while Figure 6 visualises the same frontier.

The budget sweep in Table 7 and figure 6 is the strongest result in Part 2. To make the trade-off explicit, storage paid is defined as the fraction of the full `Q4`→`Q8` storage increase incurred by a mixed model, and gap recovered is the fraction of the full `Q4`→`Q8` perplexity gap it closes:

$$\text{StoragePaid} = \frac{S_{\text{mixed}} - S_{Q4}}{S_{Q8} - S_{Q4}}, \qquad \text{GapRecovered} = \frac{PPL_{Q4} - PPL_{\text{mixed}}}{PPL_{Q4} - PPL_{Q8}}.$$

Several features of the frontier are worth noting. The low-budget region is already efficient: the matched-budget activation-aware model pays only 9.16% of the full storage increase while recovering 29.45% of the perplexity gap. The curve then steepens further. Moving from 400 MiB to 500 MiB increases storage paid by only 6.83 percentage points, but increases gap recovered from 63.37% to 83.22%. By contrast, moving from 500 MiB to 600 MiB adds a further 6.96 percentage points of storage while producing essentially no gain (83.22% versus 83.08%). The practical sweet spot is therefore not at the largest tested budget, but around the 500 MiB operating point.

The additional `Q6_K` anchor sharpens that interpretation. Uniform `Q6_K` recovers 79.19% of the `Q4`→`Q8` perplexity gap while paying 45.57% of the storage increase. The 500 MiB mixed model instead recovers 83.22% of the gap while paying only 34.59% of the storage increase. The mixed frontier is therefore not merely interpolating smoothly between `Q4` and `Q8`; at least at this point, it yields a genuinely better quality–storage operating point than the uniform intermediate quant.

Taken together, Part 2 supports four conclusions. First, tensor-level mixed GGUF assembly is practical within the existing local-inference ecosystem. Second, simple structural heuristics already form a strong baseline, especially when they protect the output projection and final blocks. Third, activation-aware selection improves on that baseline at matched budget, though the gain is modest because the heuristic baseline is already well chosen. Fourth, the budget sweep is the main empirical result: it exposes a distinctly non-linear frontier in which a large fraction of the `Q4`→`Q8` quality gap can be recovered well before the full `Q8` storage cost is paid.

5 Limitations and Validity

The study has four main limitations. First, the detailed empirical evaluation is concentrated on a single primary model family, `Qwen2.5-3B-Instruct`, so the exact shape of the observed trade-offs may differ for other architectures and scales [9, 11]. However, Part 1 was additionally sanity-checked on `Llama-3.2-1B-Instruct-GGUF`, where the same pipeline successfully performed download, orchestration, ingestion, plotting, and recommendation using configuration changes only [7]. This strengthens the claim that the benchmarking framework is largely configuration-driven across GGUF-supported model families, even though the main quantitative study remains centred on Qwen.

Second, Part 1 recommends deployments using systems metrics and quantisation rank rather than downstream task accuracy. This makes it useful for feasibility and performance selection under edge constraints, but it is not intended as a substitute for end-task evaluation. That trade-off was deliberate: the focus of Part 1 was deployment selection under constrained hardware rather than benchmark accuracy optimisation.

Third, Part 2 uses perplexity as its primary quality metric. This is reproducible and suitable for controlled comparative model analysis, but it does not fully capture instruction-following quality or human preference [6, 4]. The aim of Part 2 was therefore to study relative quality–storage behaviour under selective precision retention rather than to provide a comprehensive task-level evaluation.

Fourth, runtime benchmarking was largely single-run, so very small throughput differences should be interpreted cautiously. However, the Raspberry Pi repeatability study showed very low variance for the lower-cost configurations, especially `Q4_K_M`, and therefore supports the credibility of the main comparative conclusions. The framework also supports repeated benchmarking when higher-confidence measurement is required, at the cost of increased runtime and compute.

These limitations bound the scope of the results but do not weaken the central findings. Part 1 is best interpreted as a portable, configuration-driven deployment framework whose portability was validated across runtime profiles, operating environments, a physical edge device, and an additional model family. Part 2 is best interpreted as a research-oriented prototype with a generic GGUF mixing core, but with model-specific saliency and recipe stages evaluated in depth on one family. The Raspberry Pi study was intentionally compact because its goal was portability validation rather than a second exhaustive benchmark campaign, and the mixed-precision framework operates at whole-tensor granularity because finer-grained schemes would require kernel- and runtime-level changes outside the scope of this project.

6 Conclusion

This paper investigated edge LLM deployment as a practical systems problem and studied how much of the resulting quality–efficiency trade-off can be recovered without leaving the GGUF/`llama.cpp` ecosystem [2, 1]. The first contribution was a portable benchmarking, orchestration, and recommendation framework that measures candidate configurations on the target device, logs structured artefacts, and returns scenario-specific deployment recommendations. The second was a tensor-level mixed-precision GGUF mixer that selectively upgrades chosen tensors while keeping the model runnable in the same local-inference stack.

The results show that there is no universally best quantised deployment independent of device and workload. On the Raspberry Pi 5, the same benchmark corpus led to different recommendations for latency-first, balanced, quality-first, and maximum-context objectives, which is precisely the behaviour a practical deployment framework should support. The additional `Llama-3.2-1B-Instruct-GGUF` sanity test further showed that Part 1 is largely configuration-driven across GGUF-supported model families: the full workflow transferred with configuration changes only, rather than code changes [7].

The mixed-precision results then showed that selective precision allocation is worthwhile once a low-bit baseline has been chosen: a 500 MiB mixed model achieved better perplexity than uniform `Q6_K` while using less storage, and recovered 83.2% of the `Q4`→`Q8` perplexity gap while paying only 34.6% of the corresponding storage increase. This supports the view that useful quality recovery can be achieved without paying the full cost of uniform higher precision.

The central conclusion is therefore twofold. Edge deployment should be approached empirically, with device-specific measurement rather than static rules of thumb; and, once an efficient baseline has been selected, selective tensor-level promotion provides a practical route to recovering a large fraction of the lost quality without paying the full cost of uniform higher precision. Additional implementation details, figures, and supplementary artefacts are provided in the project repository.

References

[1] ggml-org. Gguf format specification. https://github.com/ggml-org/ggml/blob/master/docs/gguf.md, 2026. Accessed: 2026-03-27.

[2] ggml-org. llama.cpp: Llm inference in c/c++. https://github.com/ggml-org/llama.cpp, 2026. Accessed: 2026-03-27.

[3] Hugging Face. Gguf documentation. https://huggingface.co/docs/hub/gguf, 2026. Accessed: 2026-03-27.

[4] Hugging Face. Salesforce/wikitext: wikitext-2-raw-v1. https://huggingface.co/datasets/Salesforce/wikitext/tree/main/wikitext-2-raw-v1, 2026. Accessed: 2026-03-27.

[5] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. Awq: Activation-aware weight quantization for llm compression and acceleration. *Proceedings of Machine Learning and Systems*, 6:87–100, 2024.

[6] Stephen Merity, Nitish Shirish Keskar, and Richard Socher. An analysis of neural language modeling at multiple scales. *arXiv preprint arXiv:1803.08240*, 2018.

[7] Meta Llama Team. Llama-3.2-1b-instruct model card. https://huggingface.co/meta-llama/Llama-3.2-1B-Instruct, 2024. Accessed: 2026-03-27.

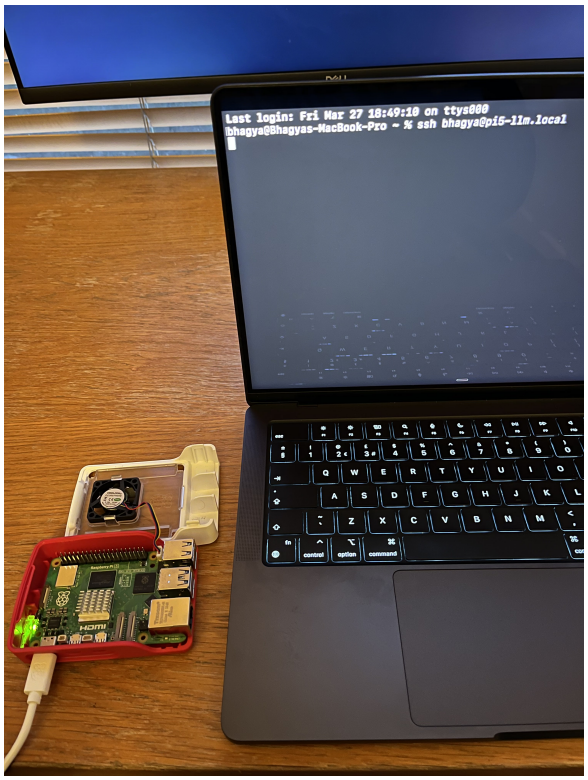
[8] PyTorch. Modules — pytorch documentation. https://docs.pytorch.org/docs/stable/notes/modules.html, 2026. Accessed: 2026-03-27.

[9] Qwen Team. Qwen2.5-3b-instruct model card. https://huggingface.co/Qwen/Qwen2.5-3B-Instruct, 2024. Accessed: 2026-03-27.

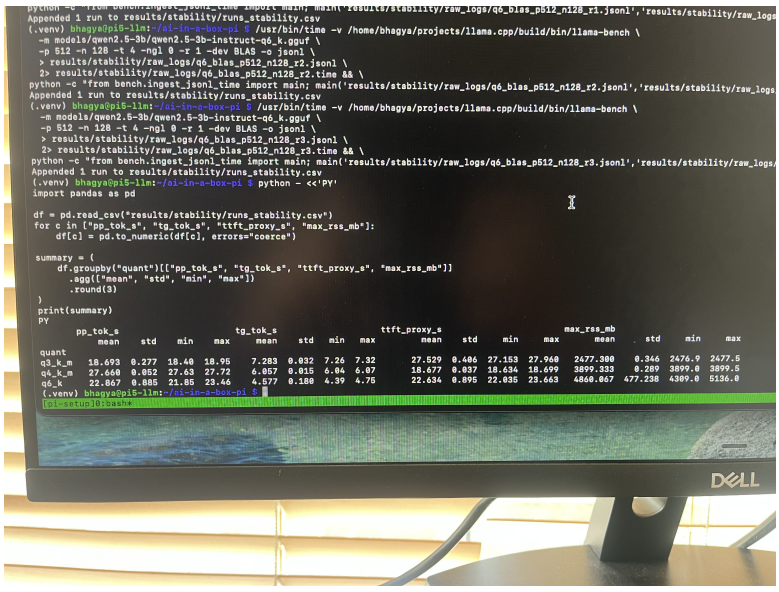
[10] Raspberry Pi Ltd. Raspberry pi 5. https://www.raspberrypi.com/products/raspberry-pi-5/, 2026. Accessed: 2026-03-27.

[11] An Yang, Baosong Yang, Beichen Zhang, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

7 Appendix: Supplementary Artefacts



(a) Raspberry Pi 5 setup used for constrained-device validation.



(b) Benchmarking Qwen2.5-3B on the Raspberry Pi 5 via SSH.

Figure 7: Supplementary photographs of the Raspberry Pi 5 setup and benchmarking. The host machine was used to orchestrate and monitor runs over SSH.

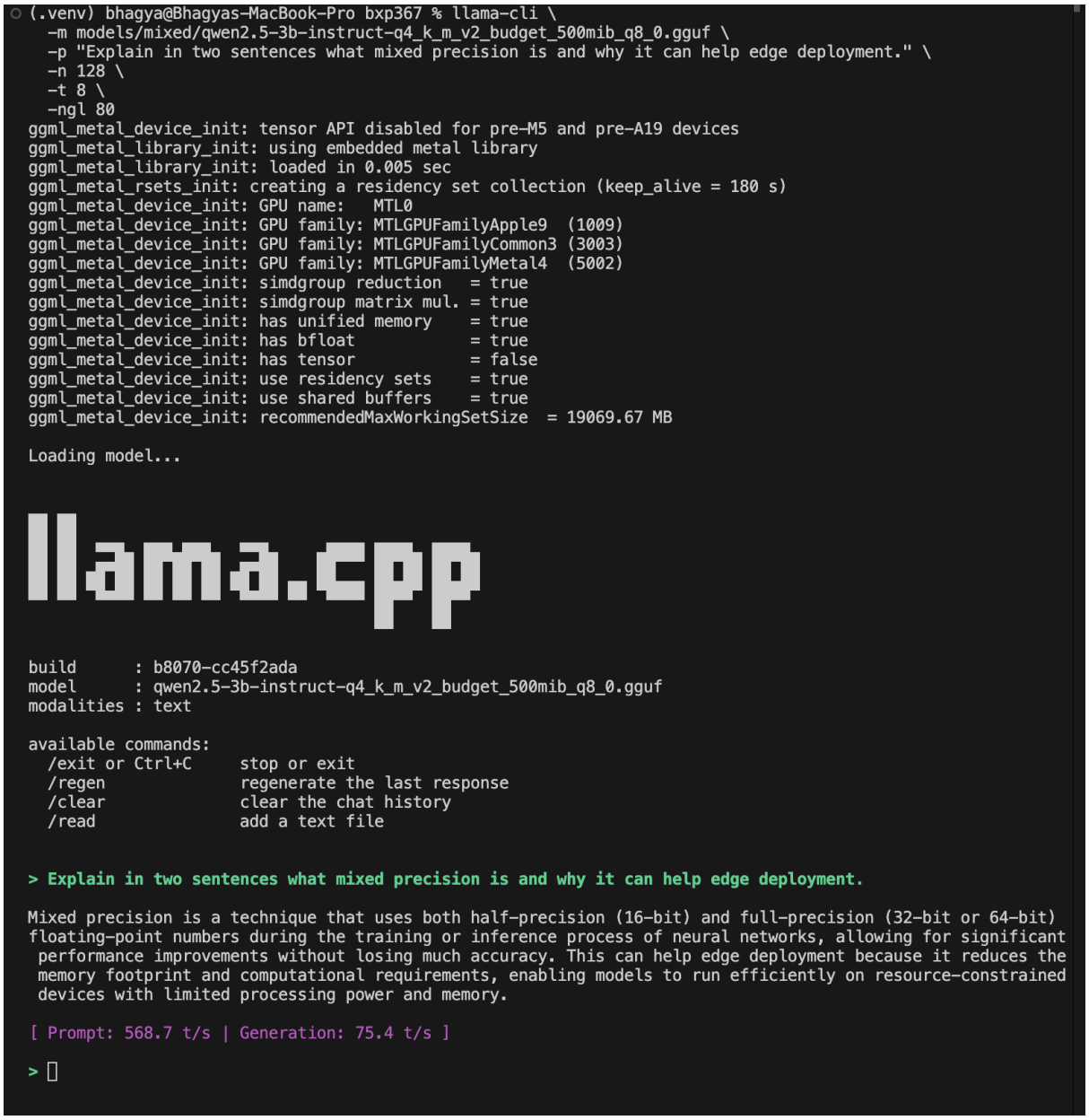


Figure 8: Supplementary screenshot showing a generated mixed-precision GGUF model executing successfully in `llama-cli`.